



API 보안 기초

API란 무엇인가?

인터넷 트래픽의 대다수를 차지하는 API

전통적 웹 특징

API 중심 특징

일반 웹 통신 vs API 통신

오늘날 서비스에서 API가 차지하는 중요성

API 보안이란 무엇인가?

API의 중요성

API가 공격 대상이 되는 이유

모의해킹 관점에서의 중요성

API란 무엇인가?

- API는 **Application Programming Interface**의 약자로, 프로그램끼리 서로 소통하는 규칙과 약속
- 쉽게 말해, 서로 다른 시스템을 연결하는 통로 역할

레스토랑 비유로 이해하기

당신이 레스토랑에 들어갑니다. (클라이언트) ↓ 메뉴판을 보고 "스테이크를 미디엄으로 주세요"라고 주문서를 씁니다. (API 요청) ↓ 이 주문서는 웨이터(API)를 통해 주방(서버)으로 전달됩니다. ↓ 주방은 주문서를 보고 실제 스테이크를 조리합니다. (데이터 처리) ↓ 완성된 스테이크는 다시 웨이터(API)를 통해 당신에게 전달됩니다. (응답 반환) ↓ 당신은 직접 주방에 들어가지 않아도 원하는 스테이크를 받을 수 있습니다.

- 손님 (클라이언트/프로그램)
 - 원하는 정보를 요청하는 주체
- 메뉴판 (API 문서)
 - 레스토랑에서 제공하는 **음식(기능)** 목록과 각 음식에 대한 **설명(요청 형식)**이 적혀 있음
 - 제공 가능한 기능과 요청 형식이 정리된 설명서
 - 예: "스테이크 - 미디엄 굽기 선택 가능"
- 주문서 (API 요청)
 - 메뉴판을 참고해 작성하는 구체적 요청
 - 예: "스테이크 1개, 굽기는 미디엄"
- 주방 (서버)
 - 주문서를 받아 실제 음식을 만드는 곳
 - 요청에 따라 데이터를 처리하고 결과를 만들어냄
- 웨이터 (API)
 - 손님과 주방 사이를 오가며 주문을 전달하고, 완성된 음식을 가져다주는 역할
 - 손님은 직접 주방에 들어가지 않고, 웨이터(API)를 통해 원하는 것을 얻음

실제 예시 - 날씨 앱

- 스마트폰(클라이언트)이 날씨 정보를 보여주려면, 기상청 서버와 직접 소통할 필요가 없음
- 기상청이 제공하는 **날씨 API**를 통해 "서울의 오늘 날씨를 알려줘"라고 요청하고, API는 서버에서 가져온 데이터를 당신의 앱에 전달

기상청 API허브



<https://apihub.kma.go.kr/>

실제 예시 - 온라인 쇼핑몰

- 당신이 상품을 구매할 때, 결제 시스템(예: 토스페이, 네이버페이)이 제공하는 **결제 API**를 호출

- 이 API는 당신의 결제 정보를 받아 안전하게 처리한 후, 결제가 완료되었다는 응답을 다시 보내줌

코어 API | 토스페이먼츠 개발자센터

토스페이먼츠 API 엔드포인트(Endpoint)와 객체 정보, 파라미터, 요청 및 응답 예제를 살펴보세요.

<https://docs.tosspayments.com/reference>

 payments 개발자센터



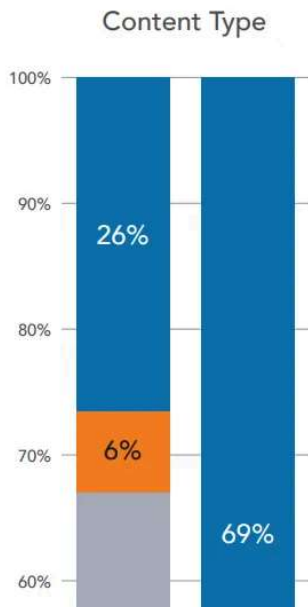
네이버페이 개발자센터

네이버페이 개발자센터를 통해 쉽고 빠르게 결제 연동을 지원해 드립니다.

 <https://developers.pay.naver.com/docs/v2/api>

인터넷 트래픽의 대다수를 차지하는 API

- Akamai 연구원에 따르면 2014년에는 HTTPS 트래픽 중 약 **47%**가 API로 분류되었으나, 2019년에는 **83%**까지 증가했고, HTML 트래픽은 **17%**로 감소
- 이후에도 API가 전체 트래픽의 대부분을 차지하며, **API 대상 공격 트래픽 또한 지속적으로 증가**



RISE OF API TRAFFIC

In 2014, Akamai researchers asked a relatively simple question: How much of the HTTPS traffic on our network is API compared to HTML? In other words, we wanted to know what portion of the traffic we see, and by extension the Internet as a whole, is content formatted for machines – some of which is triggered by human activity, and some of which is automated data exchanged behind the scenes without direct human interaction. The assumption had been that API traffic was a small portion of our traffic, but an informal analysis of our statistics revealed that API traffic accounted for 47% of all layout and data traffic we saw.

This was a major revelation – one that fueled a multitude of conversations in the past four years. We recently decided it was time to look at API traffic again, and the results were once again surprising: The traffic classified as **APIs currently accounts for 83% of all hits**, while HTML traffic has fallen to just 17%.

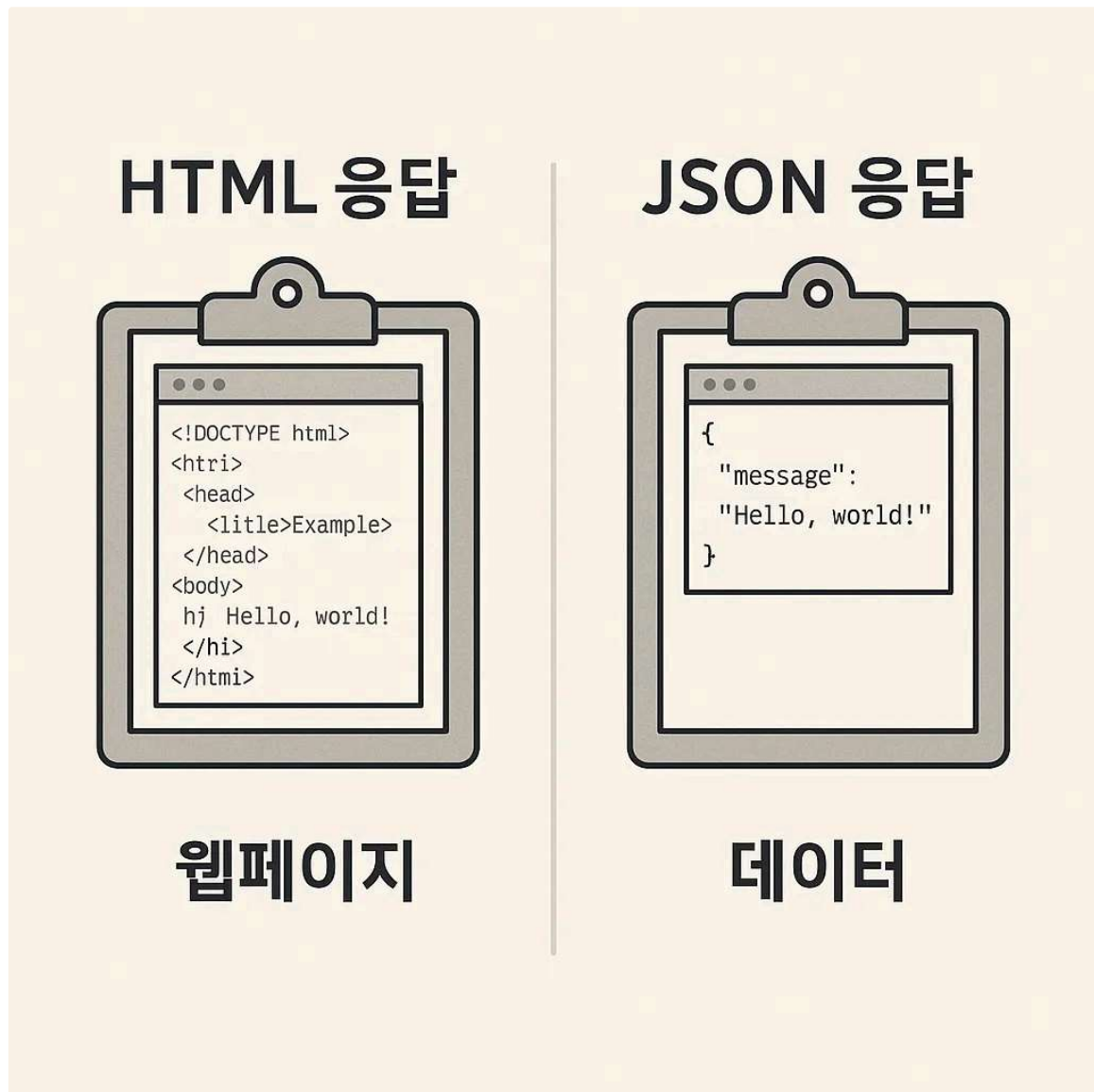
State of the Internet / Security | Retail Attacks and API Traffic (Volume 5, Issue 2) | Akamai

전통적 웹 특징

- **서버 렌더링**: 서버가 완성된 HTML을 만들어 보내므로 브라우저는 주로 표시(렌더링)만 담당
- **상태 유지**: 서버 세션 + 쿠키 기반 인증을 주로 사용
- **주요 보안 포인트**: XSS, CSRF, 입력 검증, 세션 관리(세션 고정 등)
- **구조적 특징**: 페이지 단위로 로직이 나뉘고, 과거에는 동일한 백엔드를 여러 종류의 클라이언트가 재사용하는 경우가 지금보다 적었음
- **예시**: 전통적 블로그·쇼핑몰 상품 페이지 = 서버에서 HTML 생성 → 브라우저에 전송

API 중심 특징

- **프론트엔드의 역할 확대**: 클라이언트(SPA 등)가 데이터(JSON)를 받아 UI를 조립하므로 서버는 주로 데이터 제공자 역할
- **다양한 소비자**: 웹, 모바일, 서드파티 통합 등 여러 클라이언트가 동일한 API를 호출하여 **재사용성·통합성**이 크게 증가
- **인증·권한의 다양화**: JWT(JSON Web Token), OAuth2, API 키, Bearer 토큰 등 토큰 기반 인증이 널리 쓰이며, 이로 인해 서버 측 **토큰 검증·서명·권한 체크**가 핵심
- **주요 보안 포인트**: BOLA (객체 단위 권한 검증 미흡), **Excessive Data Exposure**, **Broken Function Level Authorization** (기능 단위 권한 검증 미흡), **Mass Assignment** (대량 할당)
- **예시**: 동일한 `/api/v1/users/123` 엔드포인트를 모바일 앱, SPA, 백오피스가 함께 호출



일반 웹 통신 vs API 통신

- 전통적 웹 보안은 주로 브라우저 환경에서 발생하는 공격(예: XSS, CSRF)에 많은 비중
- API 보안은 화면보다는 데이터 보호와 권한 검증 모델을 중심으로 접근하는 것이 특징
- 다만 웹에서도 권한 검증은 여전히 핵심이고, API 역시 클라이언트 측 취약점과 무관하지 않음
 - 즉, 공통적인 보안 원칙은 같지만, 환경에 따라 특히 강조되는 위험이 다르다는 점이 중요함

구분	웹	API
주요 응답 형태	HTML / CSS / JS (완성된 페이지)	JSON / XML 등 데이터 페이로드
주요 보안 초점	사용자 입력의 안전한 렌더링(예: XSS), 세션·쿠키 안전	권한 검증(Object-Level Authorization), 데이터 노출 최소화, 토큰·인증 흐름
대표적 취약점(자주 보는 사례)	XSS, CSRF, Clickjacking, Insecure Direct Object Reference	BOLA (Broken Object Level Authorization), Excessive Data Exposure, Broken Function Level Auth, Rate limiting
공통 취약점	입력 검증, 인증 관리, 암호화, 로깅/모니터링 필요	입력 검증, 인증 관리, 암호화, 로깅/모니터링 필요
취약점 발견	브라우저에서 바로 확인 가능 → 눈으로 재현·검증 쉬움	요청/응답을 캡처·분석해야 보임 → 프록시(예: Burp)·Postman 등 도구 필수

오늘날 서비스에서 API가 차지하는 중요성

- 현대 애플리케이션은 **데이터 중심**이며, 여러 클라이언트(모바일 앱, SPA, 타사 연동 서비스)가 동일한 백엔드 API를 호출
 - SPA(Single Page Application, 단일 페이지 애플리케이션)는 웹사이트 전체를 새로고침하지 않고, 필요한 데이터만 서버에서 비동기(AJAX 등) 방식으로 가져와 현재 화면을 빠르게 갱신하는 방식
- **하나의 API 엔드포인트에 결함이 있으면**, 그 API를 사용하는 모든 클라이언트와 연동 지점이 동시에 영향을 받음 (웹·앱·파트너 서비스 모두)
- API는 내부 시스템의 핵심 데이터(개인정보, 결제 정보, 서비스 설정 등)를 직접 다루기 때문에, API 취약점은 곧바로 **데이터 유출, 권한 오용(Privilege Abuse), 서비스 기능 무단 실행**으로 이어질 수 있음
- **예시 - 구글 OTP**
 - >> 기본 계정 로그인 > 2차 인증 구글 **OTP**...
 - OTP 앱은 로컬에서 코드를 생성하지만, 사용자가 입력한 코드는 결국 **서버 검증 API**를 통해 확인
 - 이 과정에서 검증 API가 안전하지 않다면 → **인증 우회나 재사용 공격** 가능

- 예시 – OAuth 연동(구글 로그인)

1. 사용자가 “구글 계정으로 로그인” 버튼 클릭
2. 웹사이트는 구글의 **Token/UserInfo API**에 요청 → 구글은 Access Token 및 사용자 프로필을 응답
3. 서비스가 토큰과 응답을 제대로 보호·검증하지 않으면 → **인증 위조, 세션 탈취, 권한 없는 접근** 발생

- 참고) 네이버 로그인 API 명세

네이버 로그인 API 명세 - LOGIN

네이버 로그인 API 명세 네이버 로그인 API는 네이버 로그인 인증 요청 API, 접근 토큰 발급/갱신/삭제 요청 API로 구성되어 있습니다. 네이버 로그인 인증 요청 API는 여러분의 웹 또는 앱에 네이버 로그인 화면을

 <https://developers.naver.com/docs/login/api/api.md>

API 보안이란 무엇인가?

API의 중요성

- 오늘날 앱 중심 사회에서 **API는 혁신의 핵심 요소**
- 은행, 소매·운송 업계 IoT, 자율주행차, 스마트 시티까지 다양한 산업에서 활용되며, 모바일·SaaS·웹 애플리케이션의 근간을 이루고 있음
- API는 세 가지 방식으로 사용
 - **고객 대상(Customer-facing)**: 고객이 직접 사용하는 서비스 (예: 쇼핑몰 앱)
 - 취약 시 개인정보 유출·서비스 장애로 직결
 - **파트너 대상(Partner-facing)**: 파트너사가 활용하는 서비스 (예: 협력사 전용 API)
 - 인증·권한 검증이 부실하면 데이터 남용 가능
 - **내부 전용(Internal)**: 기업 내부 직원이나 개발자가 사용하는 시스템 (예: 인사·재무 관리 시스템)
 - 내부자 위협이나 잘못된 접근 통제에 의한 사고 위험

API가 공격 대상이 되는 이유

- API는 **애플리케이션 로직**과 민감한 데이터(PII, 개인식별정보 등)를 직접 노출
- 이 때문에 점점 더 **공격자의 주요 타겟**이 되고 있음
- 보안이 취약한 API는 곧바로:
 - **데이터 유출**: 개인정보, 결제·계좌 정보 탈취
 - **권한 오용**: 다른 사용자 계정 접근, 관리자 권한 상승
 - **서비스 기능 남용**: 결제 승인·주문 취소·계정 삭제 등 정상 기능을 악용
- 결과적으로 안전하지 않은 API는:
 - **기업의 서비스 신뢰도를 훼손**하고
 - **규제·법적 리스크를 증가**시키며
 - **비즈니스 혁신 속도를 심각하게 저해**함

모의해킹 관점에서의 중요성

- **광범위한 공격 표면 (Attack Surface)**
 - API는 모바일 앱, 웹 서비스, IoT 기기, 서버 간 통신 등 **거의 모든 서비스의 중심**에 존재
 - 공격자는 노출된 API 엔드포인트를 직접 분석·자동화 도구로 스캔하여 공격 기회를 탐색
 - 따라서 모의해킹 과정에서 API는 **중요한 점검 대상**
- **보안 핵심 기능의 집중 지점**
 - API는 **인증(Authentication)**, **권한(Authorization)**, **데이터 흐름(Data Flow)** 같은 보안 핵심 기능을 처리
 - 작은 취약점도 곧바로:
 - 세션 탈취 → 전체 계정 장악
 - 권한 검증 미비 → 관리자 권한 획득